Deloitte.

ABAP on HANA Tools

Deloitte Technology Academy (DTA)

Agenda

Topic	Descriptions	Duration
Overview of Custom ABAP Code Migration to SAP Business Suite powered by SAP HANA	Custom ABAP Code migration to SAP Business Suite Powered by SAP HANA, Prepare Custom Code for SAP HANA, Functional Corrections for Existing ABAP Code—Details and Examples, Static Checks I, Static Checks NW 740 SP5, and ABAP Test Cockpit and Static Checks	4 hours
Optimize Custom Code for SAP HANA	Tooling, Classic Golden SQL Rules, Shift in Priorities of Classic Golden SQL Rules for SAP HANA, Static Performance Checks, Monitoring Tools, Aggregation and SQLM data records, DB Performance Profile of the Business Process, Lab Preview 740 SP8—Time Series and SQL Trace, SQL Monitor, Guided Performance Optimization, Key Takeaways	

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Learning Objectives

By the end of this session, you will be able to:

- Describe the impact of migration to SAP Business Suite powered by SAP HANA on custom ABAP development
- Prepare your custom Code for SAP HANA
- Optimize custom code for SAP HANA

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Overview of Custom ABAP Code Migration to SAP Business Suite powered by SAP HANA

Impact of Migration on Custom ABAP Development

</>

The following are the key questions to ask when migrating your code to SAP Business Suite powered by SAP HANA.

- Which of my ABAP code must be changed to avoid potential functional issues?
- Which of my ABAP code has to be optimized to ensure good performance?
- Which of my main business processes have the potential for HANA so that I can speed up those massively?
- How can I find all this ABAP code easily and do the adaption efficiently?

```
* look for goods receipt greater goods issue
*
SELECT * FROM EKET
WHERE EBELN IN S_EBELN
ORDER BY EBELN EBELP EINDT ETENR.
  CHECK EKET WEMNG > EKET WAMNG.
  SELECT SINGLE * FROM EKKO WHERE EBELN = EKET-
                              EBELN AND RESWK NE SPACE.
  CHECK SY-SUBRC EQ 0.
  check eket ebeln ne xerror-ebeln or
        eket-ebelp ne xerror-ebelp.
  move-corresponding eket to xerror.
  append xerror.
ENDSELECT.

sort xerror by ebeln ebelp eindt etenr.
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

Functional Corrections for Existing ABAP Code

Which of my ABAP code must be changed to avoid potential functional issues?

- In general, existing ABAP code runs on SAP HANA as before
- However, if ABAP code relies on technical specifics of the previous database, ABAP code changes might be necessary

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Technical Changes

Technical changes with SAP HANA that may affect existing DB specific ABAP code

Technical Change	Details and Examples—Effect on DB Specific ABAP Code
DB migration	Each DB has specific features and unique technical behavior DB specific code may rely on these features of the used database
SAP HANA architecture	• Column based architecture—As a consequence e.g., secondary DB indexes are less important • DB specific code may rely on the existence/usage of certain DB indexes
Depooling/Declustering	• During the migration to SAP HANA most pool and cluster DB tables are transformed to transparent DB tables (depooling/declustering) so that the tables can be used in analytic scenarios • DB specific code may rely on the technical specifics of pool and cluster tables

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Technical Change and Recommended Functional Corrections

Technical Change	Details and Examples—Effect on DB Specific ABAP Code	Recommended Solution
DB migration	Native SQL, Hints, ADBC which uses DB specific features what is ADBC? abap database connectivity	Use Open SQL if possible, and adapt native SQL and DB hints if necessary

Example for native SQL

```
Exec SQL.
   SELECT ERA, SHERA, STDATE INTO :JPERA
   FROM LDERA
   WHERE LANG =:SY-LANG
     AND STDATE = ( SELECT MAX(STDATE)
                    FROM LDERA
                    WHERE LANG = :SY-LANGU
                     AND STDATE <= :INP )
ENDEXEC.
```

Example for a DB hint

```
SELECT SINGLE la1
     FROM ABC
     INTO l_wa
     WHERE plnnr = a
     AND vornr LIKE b
     %_HINTS ORACLE ‘index(ABC “ABC~XCYZ”)’.
```

Hints are used for calling the secondary index.

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Technical Change and Recommended Functional Corrections (Cont.)

Technical Change	ABAP Code Which Must be Checked/Corrected	Recommended Solution
mostly db will sort according to primary key by its own but don't rely on it. SAP HANA architecture	Transparent DB tables: • Relying on not guaranteed/implicit sorting of DB query results • (Rare problem) Analysis of technical DB index information using function modules 'DB_EXISTS_INDEX', 'DD_INDEX_NAME'	Recommendation: • Never rely on any implicit sorting—no robust code! Always use ORDER BY ... or SORT ... BY to sort query result sets explicitly • Try to get rid of DB index analysis
pool/cluster tables are converted to transparent tables in the s/4 hana. Depooling/ Declustering	Pool/Cluster tables: • Relying on not guaranteed/implicit sorting of DB query results • Rare problem) Direct access to physical clusters/pools	Recommendation: • Never rely on any implicit sorting—no robust code! Always use ORDER BY ... or SORT ... BY to sort query result sets explicitly • Get rid of direct access to physical clusters/pools

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Functional Corrections—Details and Examples

Example for ABAP code which relies on sorted DB query results

```
Select * from ekpo
  into table lt_ekpo
  for all entries in lt_ekko
  where ebeln = lt_ekko-ebeln.

read table lt_ekpo into l_ekpo
 with key ebeln = ebeln ebelp = ebelp
 Binary search.
```

→

```
Select * from ekpo
  into table lt_ekpo
  for all entries in lt_ekko
  where ebeln = lt_ekko-ebeln.
  order by primary key.
read table lt_ekpo into l_ekpo
 with key ebeln = ebeln ebelp = ebelp
 Binary search.
```

Read Table…BINARY SEARCH works only for a correctly sorted internal table.
But no explicit sorting of the query result is done
-> Error in code which may work by chance because of DB Index usage

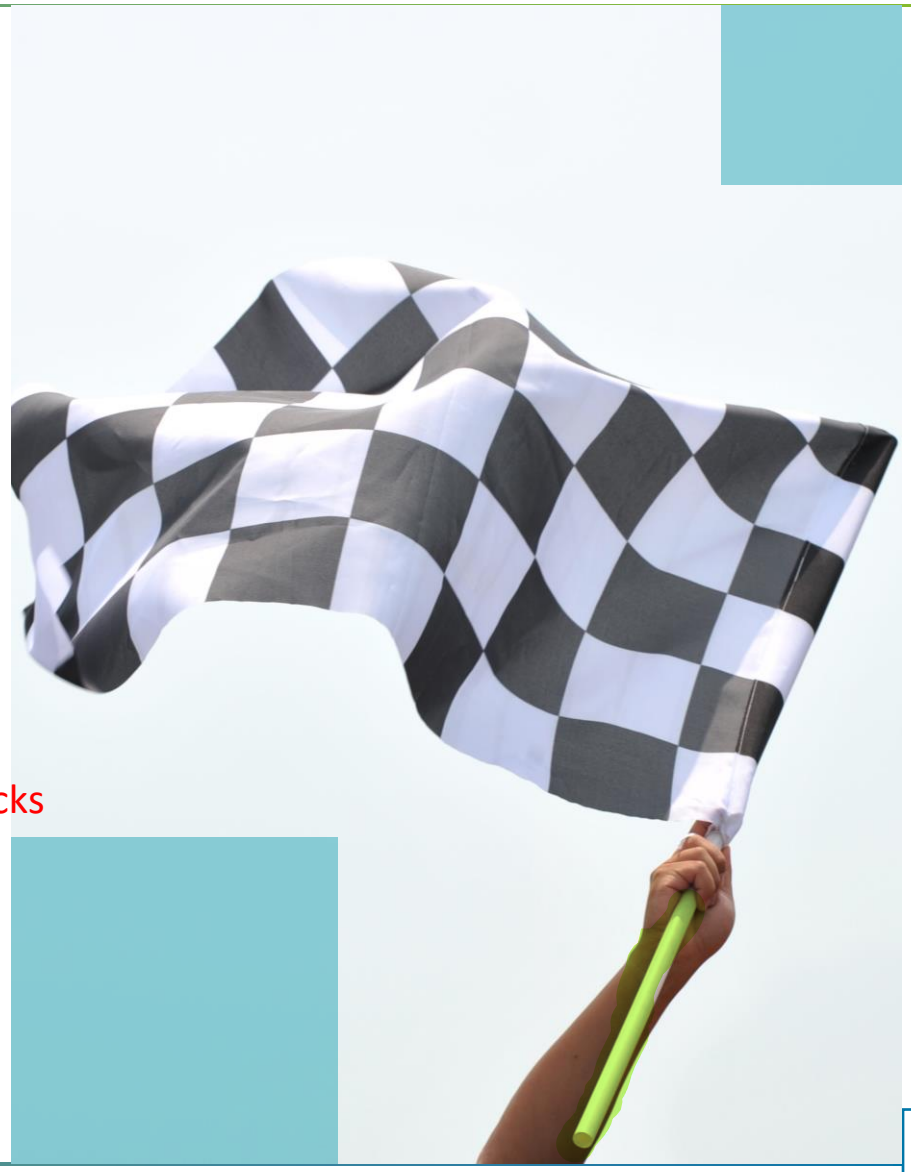
Correction
Add explicit sorting by adding ORDER BY to the SELECT or by using an ABAP SORT

Copyright © 2022 Deloitte Development LLC. All rights reserved.

ABAP Test Cockpit and Static Checks

'ATC' is also a tcode for atc tool

- In general, existing ABAP code runs on SAP HANA as before and only DB specific ABAP code must be analyzed
- New static Code Inspector Checks and Runtime Checks are available which help to find the code that must be analyzed
- ABAP Test Cockpit (ATC) is the new standard tool for static ABAP code checks
- ATC can be used now to prepare the custom ABAP code for HANA and to improve the general quality of the code
- ATC availability starts with NW 702 SP12/NW 731 SP5. In older releases, the Code Inspector can be used



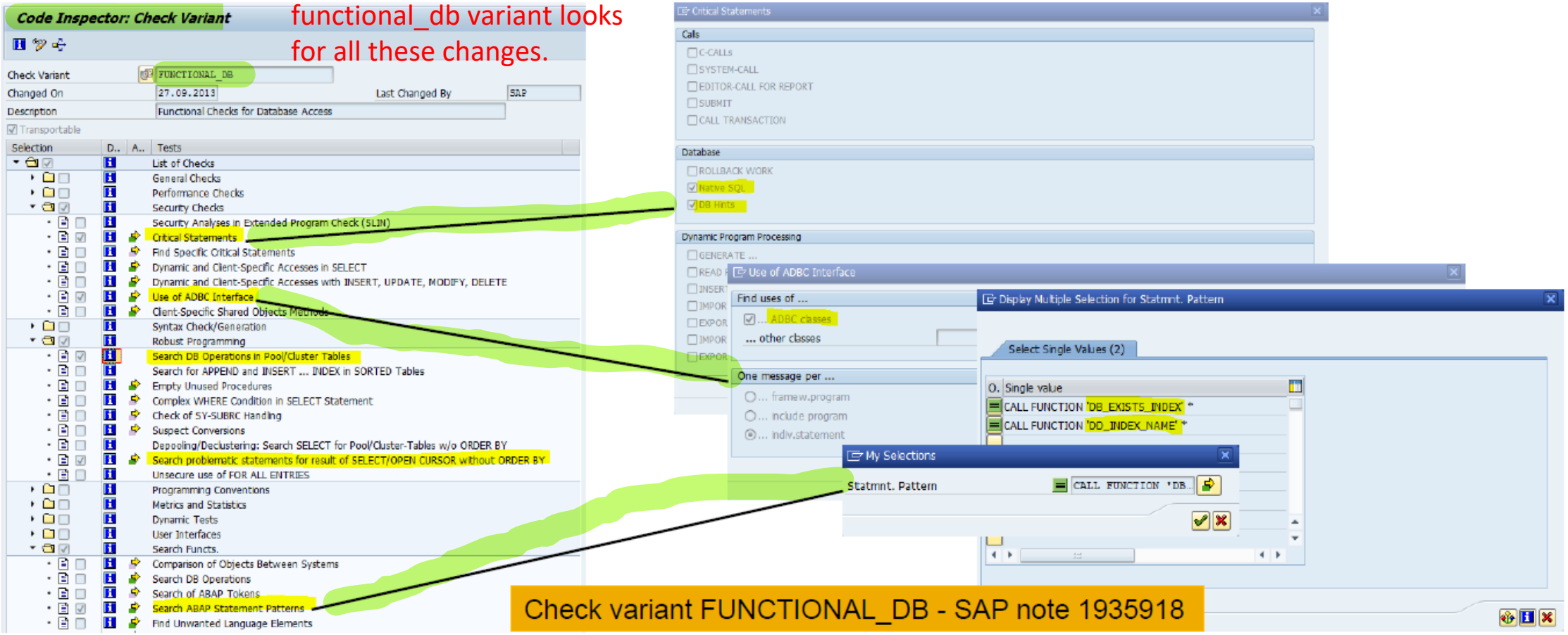
atc has three variants for three different types of functional changes/ checks that we need to do. i.e. db migration, hana architecture, depooling/ declusturing

tcode for seeing the variants is 'Sci'.

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Functional Corrections for Existing ABAP Code—Static Checks

For all main functional error categories, there are code inspector checks to find the issues easily



Copyright © 2022 Deloitte Development LLC. All rights reserved.

Functional Corrections—Static Checks NW 740 SP5—Details

"Search problematic statement, without ORDER BY" check

- Preferred check to find code that relies on sorted DB content
- Searches for statements like READ BINARY SEARCH, DELETE ADJACENT DUPLICATES, ... accessing unsorted DB content
- Works for transparent and pool/cluster DB tables with a low false positive ratio

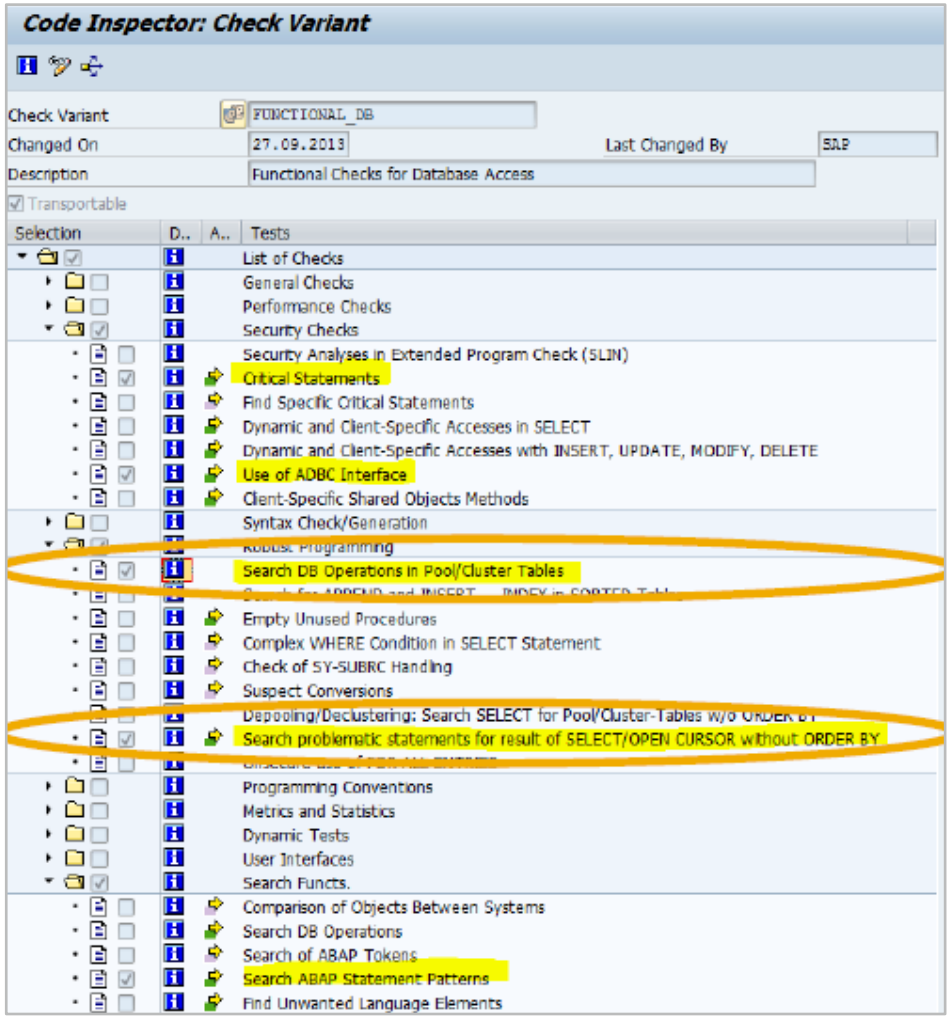
"Search DB operations in Pool/Cluster tables" check

- Searches for DB operations on the physical pool/cluster DB tables
- Check is available starting with 740 SP5 or via note 1920165

Recommendation

- Run both checks for your custom code
- Replace access to physical pool/cluster DB tables
- Concentrate on findings (priority 1 and 2) of the "Search problematic statement...without ORDER BY" check

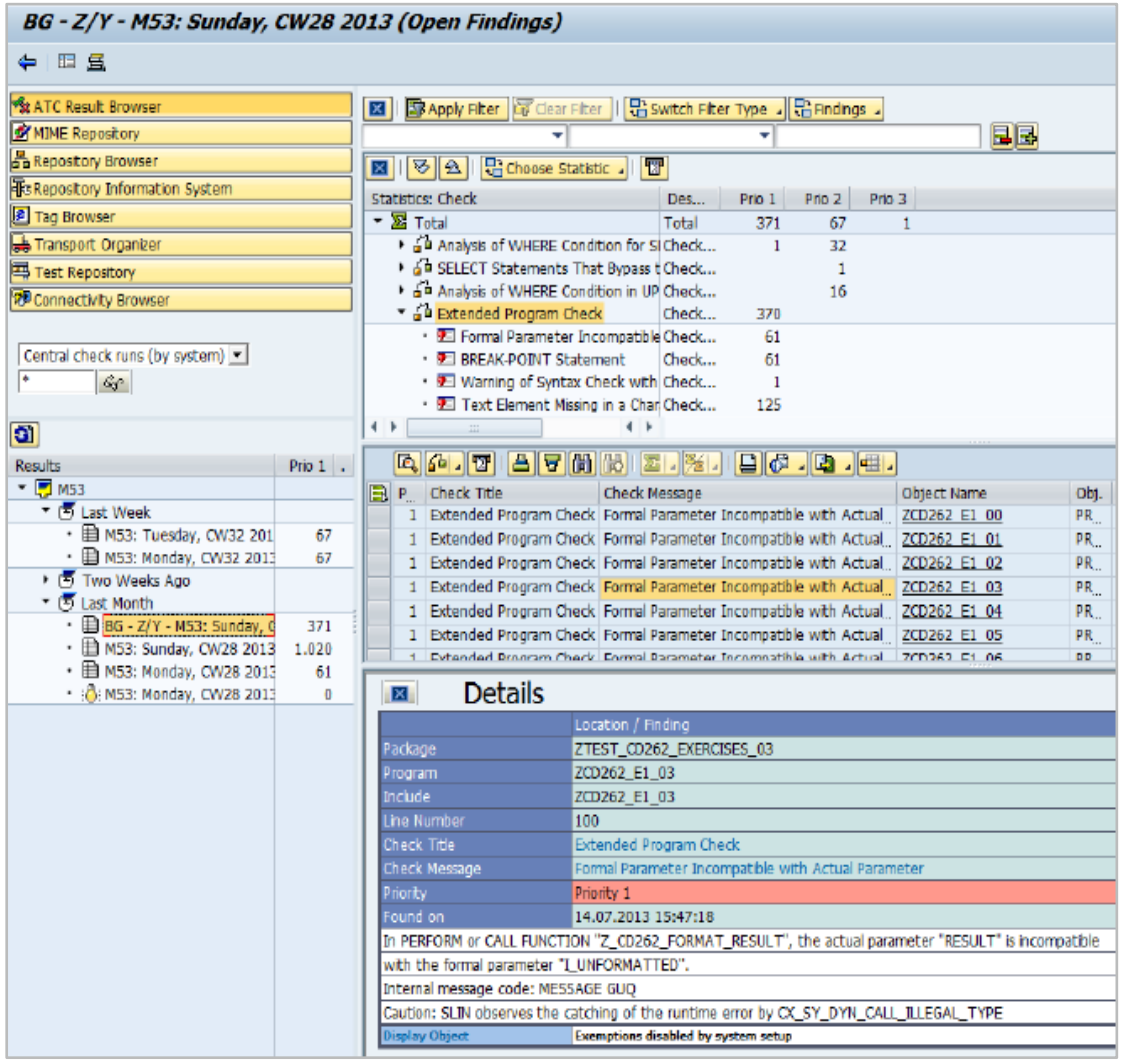
See Appendix for corresponding static checks in NW 740 SP2 — SP4



Copyright © 2022 Deloitte Development LLC. All rights reserved.

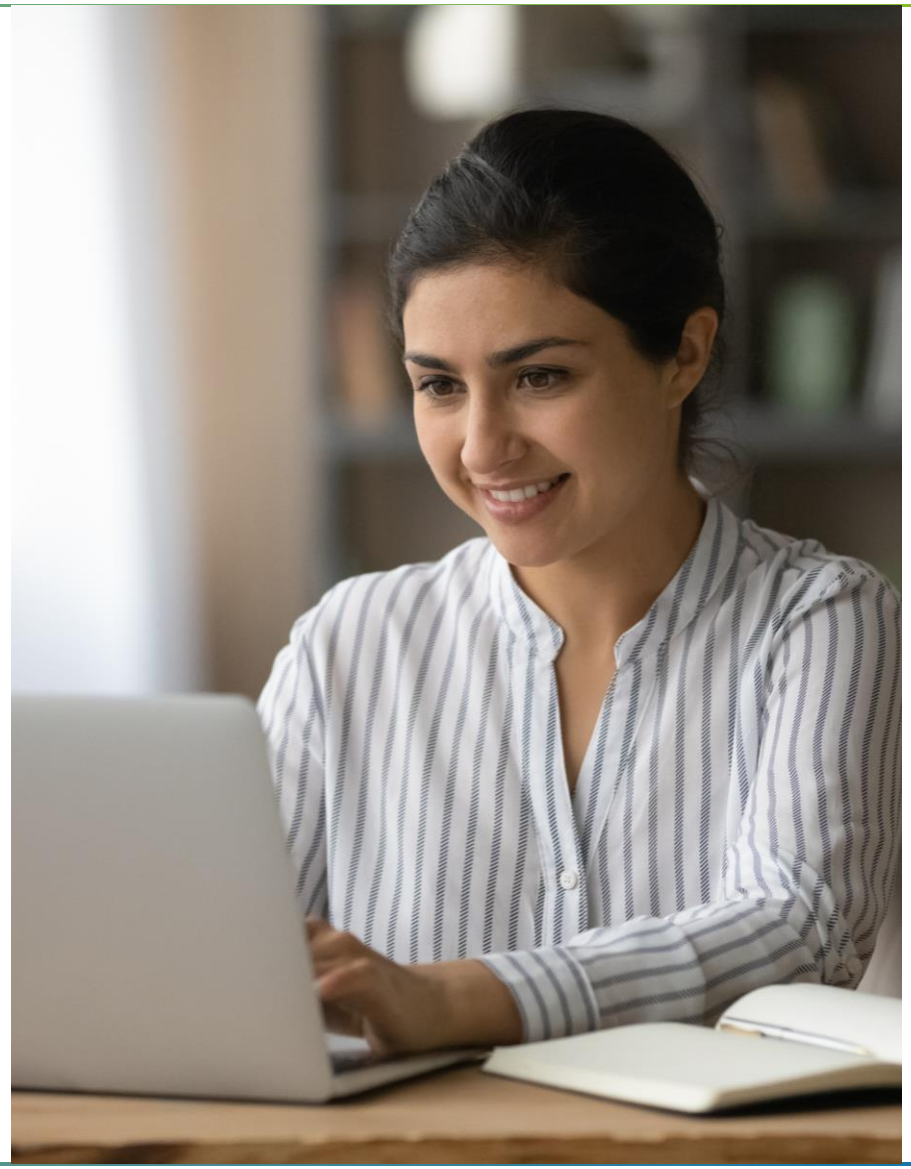
ABAP Test Cockpit—Run Your Code Inspector Checks in a New Tool

- ABAP Test Cockpit (ATC) is a framework for running static checks on ABAP objects to find functional, usability, performance, security, and bugs
- ABAP Test Cockpit is based on code inspector and adds new quality gates, quality processes, and better usability
- Lab Preview: Starting with NW 740 SP8, the developer role of the ABAP Test Cockpit is fully integrated into ABAP development tools



Copyright © 2022 Deloitte Development LLC. All rights reserved.

Optimizing Custom Code for SAP HANA



How to Optimize Custom Code for SAP HANA?

How can I find the ABAP code which needs to be optimized, or which has the potential for massive acceleration using code push down to SAP HANA?

- In general, no changes are necessary if your SQL code follows the golden Open SQL rules
- Use static checks to find SQL patterns that violate the golden Open SQL rules
- Add runtime performance data from production to rank the findings and to find potential for massive acceleration

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Classic Golden SQL Rules

Icon	Rule	Details/Examples
	Keep the result sets small	• Do not retrieve rows from the database and discard them on the application server using CHECK or EXIT, e.g., in SELECT loops • Make the WHERE clause as specific as possible
	Minimize amount of transferred data	• Use SELECT with a field list instead of SELECT * in order to transfer just the columns you really need • Use aggregate functions (COUNT, MIN, MAX SUM, AVG) instead of transferring all the rows to the application server
	Minimize the number of data transfers	• Use JOINs and/or sub-queries instead of nested SELECT loops basically avoid using too many select statement. • Use SELECT…FOR ALL ENTRIES instead of lots of SELECTs or SELECT SINGLES • Use array variants of INSERT, UPDATE, MODIFY, and DELETE
	Minimize the search overhead	Define and use appropriate secondary indexes
	Keep load away from the database	• Avoid reading data redundantly • Use table buffering (if possible) and do not bypass it • Sort data in your ABAP programs

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Shift in Priorities of Classic Golden SQL Rules for SAP HANA

Icon	Rule	Details/Examples
	Keep the result sets small	• Do not retrieve rows from the database and discard them on the application server using CHECK or EXIT, e.g., in SELECT loops • Make the WHERE clause as specific as possible
	Minimize amount of transferred data	• Use SELECT with a field list instead of SELECT * in order to transfer just the columns you really need • Use aggregate functions (COUNT, MIN, MAX SUM, AVG) instead of transferring all the rows to the application server
	Minimize the number of data transfers	• Use JOINs and/or sub-queries instead of nested SELECT loops • Use SELECT…FOR ALL ENTRIES instead of lots of SELECTs or SELECT SINGLES • Use array variants of INSERT, UPDATE, MODIFY, and DELETE
	Minimize the search overhead	Define and use appropriate secondary indexes
	Keep load away from the database	• Avoid reading data redundantly • Use table buffering (if possible) and do not bypass it • Sort data in your ABAP programs

Shift in priorities for SAP HANA

=

\+ Avoid selecting large numbers of not required columns

\+
- Prefer array operations for INSERT, UPDATE, and DELETE when changing many records
- Avoid nested SELECT loops

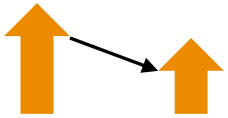
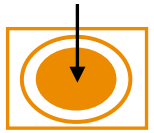
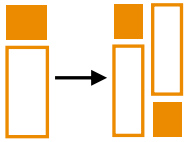
– In most cases, SAP HANA does not require secondary indices

–
- Keep unnecessary load away from the database
- Code pushdown for data-intensive calculations to benefit from SAP HANA

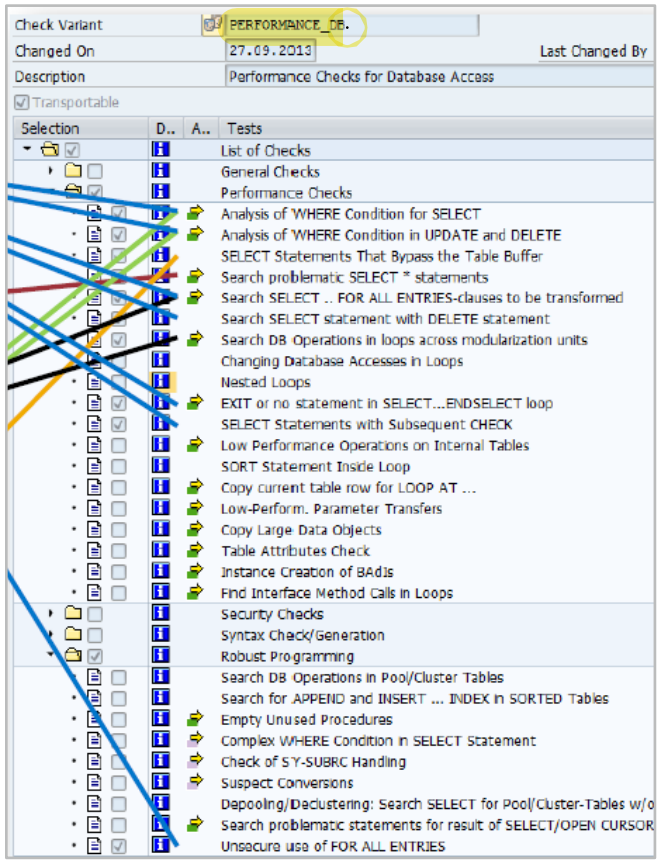
Copyright © 2022 Deloitte Development LLC. All rights reserved.

Static Performance Checks

go to sci and search for performance_db to see things in the image .

Icon	Rule	Details/Examples
	Keep the result sets small	• Do not retrieve rows from the database and discard them on the application server using CHECK or EXIT, e.g., in SELECT loops • Make the WHERE clause as specific as possible
	Minimize amount of transferred data	• Use SELECT with a field list instead of SELECT * in order to transfer just the columns you really need • Use aggregate functions (COUNT, MIN, MAX SUM, AVG) instead of transferring all the rows to the application server
	Minimize the number of data transfers	• Use JOINs and/or sub-queries instead of nested SELECT loops • Use SELECT…FOR ALL ENTRIES instead of lots of SELECTs or SELECT SINGLES • Use array variants of INSERT, UPDATE, MODIFY, and DELETE
	Minimize the search overhead	Define and use appropriate secondary indexes
	Keep load away from the database	• Avoid reading data redundantly • Use table buffering (if possible) and do not bypass it • Sort data in your ABAP programs

All golden SQL rules are covered by static checks. Check the appendix for the release where the checks are available

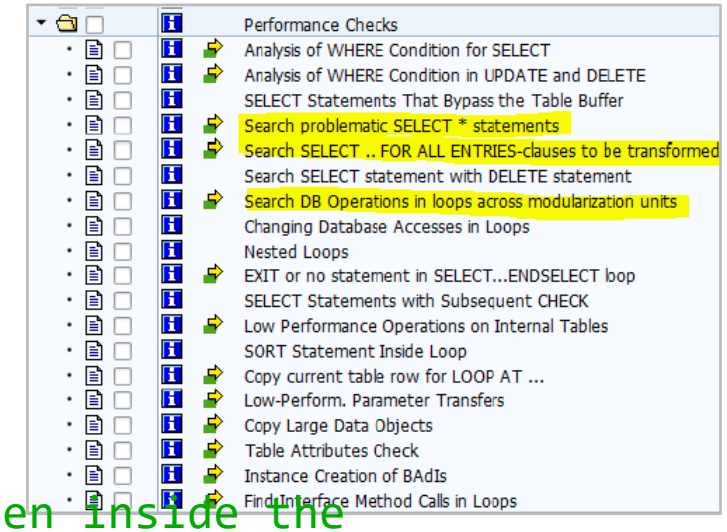


Copyright © 2022 Deloitte Development LLC. All rights reserved.

New Static Performance Checks

Minimize the amount of transferred data and the number of data transfers

- SELECT * (used column vs. selected columns)
- Nested SELECT statements (across modularization units)
- SELECT + SELECT FOR ALL ENTRIES which can be transformed into JOIN or SUBQUERY




```
Select single * from mara
  into wa_mara
  Where matnr = i_matnr
If sy-subrc = 0.
```

SELECT * check
No field used—Pure existence check

select statements shouldn't be written inside the loops

```
loop at it_mara into mara.
  perform mara_exist_check using mara-matnr
                                 changing rc.
endloop.
form mara_exist_check using i_matnr type
                    mara-matnr changing rc
select single * from mara
  into wa_mara
  where matnr = i_matnr.
if sy-sybrc =0.
```

Nested SQL check SELECT in LOOP
(across modularization unit)

SELECT + SELECT FOR ALL ENTRIES
transform into JOIN or SUBQUERY

```
SELECT vbeln FROM vbak INTO TABLE lt_vbak1 WHERE...

SELECT vbeln FROM vbpa INTO TABLE lt_vbpa FOR ALL
                                ENTRIES IN lt_vbak1
    WHERE...
      vbeln = lt_vbak1-table_line
```

Copyright © 2022 Deloitte Development LLC. All rights reserved.

New Static Performance Checks (Cont.)

same thing as the previous page

```
Select single * from mara
  into wa_mara
  Where matnr = i_matnr
If sy-subrc = 0.
```

SELECT * check
No field used—Pure existence check

```
loop at it_mara into mara.
  perform mara_exist_check using mara-matnr
                                changing rc.
endloop.
form mara_exist_check using i_matnr type
                   mara-matnr changing rc
select single * from mara
  into wa_mara
  where matnr = i_matnr.
if sy-sybrc =0.
```

Nested SQL check SELECT in LOOP
(across modularization unit)

if you are using for all entries, the first table shoudl never be empty.

```
SELECT vbeln FROM vbak INTO TABLE lt_vbak1 WHERE...

SELECT vbeln FROM vbpa INTO TABLE lt_vbpa FOR ALL
                              ENTRIES IN lt_vbak1
    WHERE...
      vbeln = lt_vbak1-table_line
```

SELECT + SELECT FOR ALL ENTRIES
transform into JOIN or SUBQUERY

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Monitoring Tools

tools for checking if your code quality is good. performance check for your code basically

Use monitoring and trace tools to measure the usage of ABAP and SQL in your productive system

- Use Usage Procedure Logging (UPL) to separate unused code (dead code)
- Use the standard performance trace tools (ST05, SAT, etc.,) to analyze your core business processes
- Find the performance hotspots with the SQL Monitor

Monitoring tools

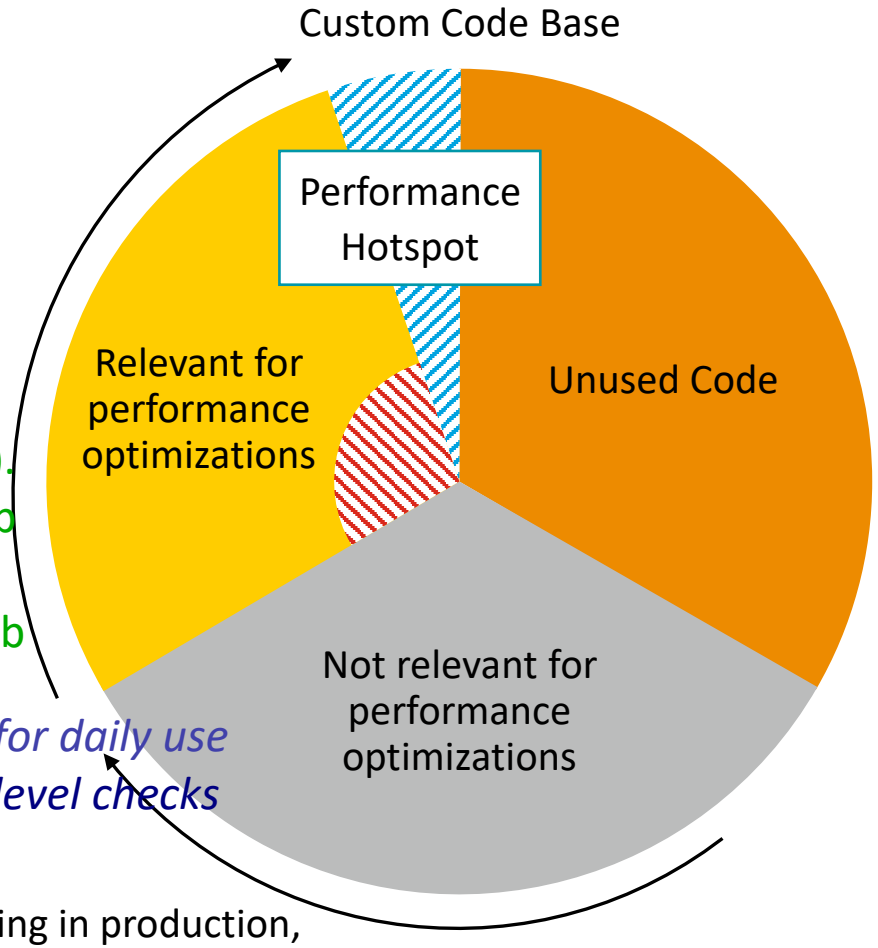
1. Performance trace (ST05). analysis related to only db
2. Runtime analysis (SAT). analysis related to both db and app layer. old tcode for it is se30.
3. SQL monitor *for upgrade level checks*

1 and 2 are for daily use

three inbuild tools to check your code

1. ATC *for upgrade level checks*
2. Code inspector *for daily basis*
3. extrended program checks *for daily use*

these tools only check the program not how it behaves after it is executed. monitoring tools will help you to understand how your program actually behaves.



Covered by SQL Monitor

After a week/month of running in production, quarter-end, or year-end processes may not be covered

Copyright © 2022 Deloitte Development LLC. All rights reserved.

SQL Monitor Overview *SQLM*

SQL Monitor collects performance data for each and every SQL executed in an ABAP application

SQL Monitor can run in a productive system without disturbing business processes (performance overhead <3%)

SQL Monitor allows linking the monitored SQL to the driving business process

SQL Monitor can be used in the “old” productive system before doing the HANA migration

SQL Monitor availability begins with NW 700

Copyright © 2022 Deloitte Development LLC. All rights reserved.

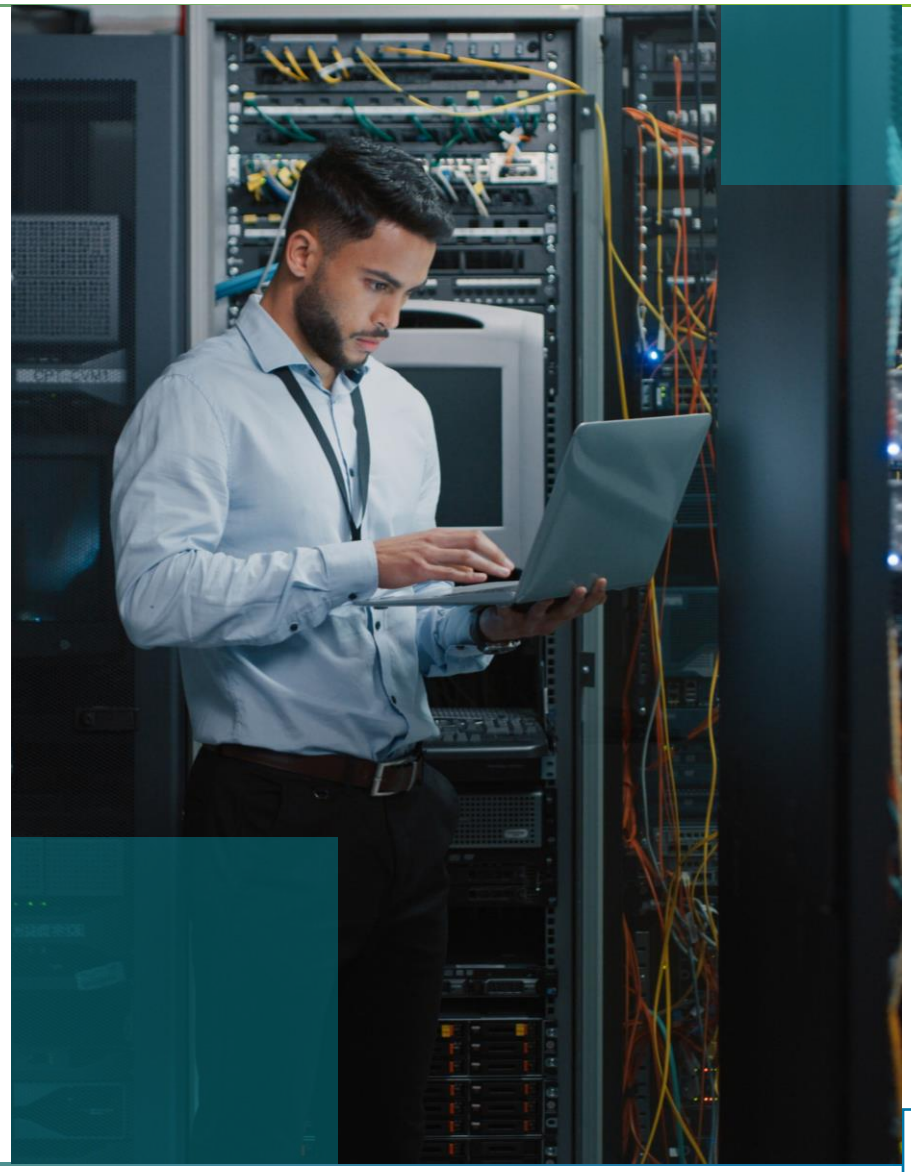
Using SQL Monitor for Optimization: Example

A productive ERP system with 6000 concurrent users

Two weeks in this system means

- 50k different business processes
- 130k different ABAP SQL statements
  - (>15% in customer code)
- 13B SQL requests (~1B/day)
- 140B records (~10B/day) read or changed

Where to start an optimization?



Copyright © 2022 Deloitte Development LLC. All rights reserved.

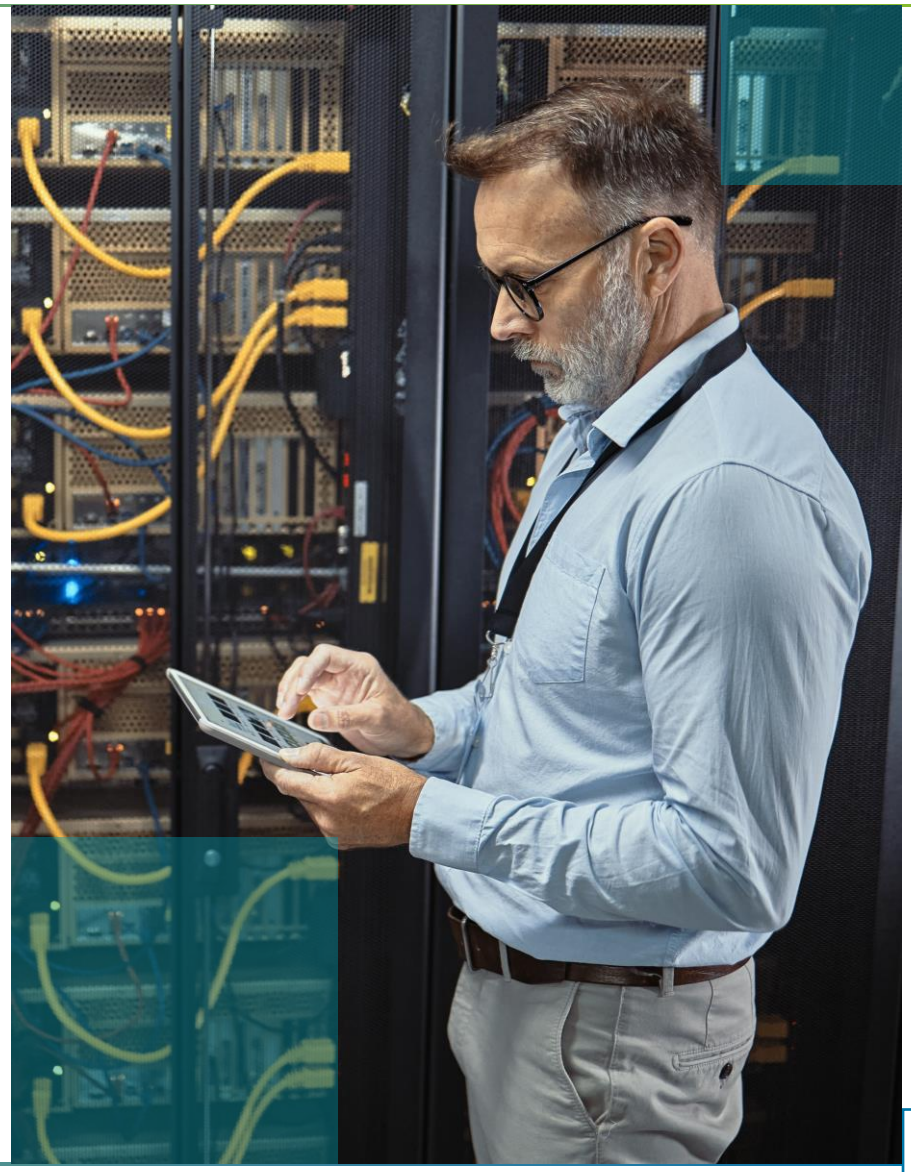
Using SQL Monitor Optimization: Example (Cont.)

The new **SQL Monitor** allows you to get performance data for each and every SQL execute in production

Answer questions like

- What are the most expensive and most frequently executed SQLs?
- Which SQL reads/writes millions of records?
- What is the SQL profile of my business process VA01?

Total transparency of the SQL running in production!



Copyright © 2022 Deloitte Development LLC. All rights reserved.

SQL Monitor—Aggregation and SQLM Data Records

SQL Monitor: Top 163 Requests (aggregated)

Application Help

Tot. DB E	Total DB Time	Total Time	DB T...	Total Records	DB Mn T	Sessi...	Request Type	Request Entry Point								
23.859.858	14.053.630,983	15.193.902,158	92,50	286.126.590	0,589	318	Batch Job	ZSQLM_TEST11								
14.684.918	86.842.478,154	148.638.736,9...	58,43	100.280.875	5,914	53	Batch Job	ZSQLM_TEST	126	9.289,7...	0,090	19,937	100.000	0	6,829	750,070
7.462.676	43.050.292,072	43.674.577,654	98,57	36.813.240	5,769	52	Batch Job	ZSQLM_TEST3	6	9.347,8...	0,381	16,349	100.000	0	4,933	650,905
7.189.218	44.820.191,834	52.344.505,931	85,63	48.028.830	6,234	640	Remote Func...	ZSQLM_CALLBACK_RFC	33	15.006,...	0,310	55,573	100.000	0	6,681	750,962
618.893	1.137.344,454	1.165.832,179	97,56	6.993.684	1,838	9	Submit Report	ZSQLM_TEST11	3	547.32...	0,340	748,489	75.000	0	11,300	889,157
430.539	1.997.205,874	2.032.910,598	98,24	2.129.610	4,639	3	Submit Report	ZSQLM_TEST3	6	3.613,6...	0,393	8,855	100.000	0	4,946	651,934

Analysis by Request

- Displays SQL performance data on request level
- Includes work process times of requests (740 SP5)

SQL Monitor: Top 200 source positions (aggregated)

Application Help

DB Executions	Total DB Time	Total Records	DB Mn T	Obj.	Object Name	Include Name	Include Li	ABAP Source Cod						
24.468.644	15.110.875,066	24.318.644	0,618	PROG	ZSQLM_TEST11	ZSQLM_TEST11	68	INSERT ZSQLM_T						
1.672	10.988,791	0	6,572	FUGR	SPOA	LSPOAF03	148	SELECT SINGLE * FROM TSPOPTIONS WHERE SPOPTION = '...	1	9.870,3...	0,309	241,371	0	0
29.843.783	173.313.289,5...	2.054.929	5,807	PROG	ZSQLM_TEST3	ZSQLM_TEST3	59	SELECT SINGLE * FROM ZSQLM_TEST_ORDER INTO WA WH...	5	974,944	0,379	7,090	1	0
2.090	2.623,208	0	1,255	FUGR	SPOA	RSPOTOOL	186	CALL 'RSPO_CACHE_CONTROL' ID 'AREA' FIELD 'TOOLS' ID '...	1	523,885	0,416	11,828	0	0
12.309	15.188,232	448	1,234	PROG	SAPMSSY2	RSBTCTRC	136	COMMIT WORK.	2	472,882	0,289	6,875	1	0
507	1.373,739	0	2,710	CLAS	CL_SQL_STATEMENT	CL_SQL_STATEMENT==...	22	CALL 'C_DB_FUNCTION' ID 'FUNCTION' FIELD 'DB_SQL' ID 'FC...	13	422,513	0,181	19,517	0	0

Analysis by Source Position

- Displays SQL performance data by source position
- Includes ABAP Source Code Fragment (740 SP5)

SQL Monitor: Top 200 records, not aggregated

Application Help

DB Exctns	Total DB Time	Records	DB Mn T	Table Names	SQL Operation	Obj.	Object Name	Include Name				
8.024	7.554,616	8.024	0,942	ALTSTLOD	UPDATE (Open S...	PROG	RSDSDEFLOAD	RSDSDEFLOAD				
6.360	4.401,385	1.590	0,692	ZSQLM_TEST_US...	SELECT, FOR ALL...	PROG	ZSQLM_TEST11_OPT	ZSQLM_TEST11_OPT	35	SELECT A FROM ZSQLM_TEST_USERS INTO TABLE ITAB FOR...	Batch Job	ZSQLM_T
5.724	2.673,153	17.172	0,467	ZSQLM_TEST_T...	FETCH (Open SQ...	PROG	ZSQLM_TEST1	ZSQLM_TEST1	53	FETCH NEXT CURSOR C2 INTO WA2.	Batch Job	ZSQLM_T
5.724	2.614,717	17.172	0,457	ZSQLM_TEST_T...	FETCH (Open SQ...	PROG	ZSQLM_TEST1	ZSQLM_TEST1	53	FETCH NEXT CURSOR C2 INTO WA2.	Batch Job	ZSQLM_T
5.719	43.541,968	5.719	7,614	TDEVC	Load Buffer (Ope...	CLAS	CL_SATC_AC_NAMESP...	CL_SATC_AC_NAMESPACE_REGISTR...	155	SELECT SINGLE NAMESPACE FROM TDEVC INTO NAMESPACE...	Submit Report	SATC_CI
5.300	6.896,637	530.000	1,301	ZSQLM_TEST_T...	SELECT, FOR ALL...	PROG	ZSQLM_TEST2	ZSQLM_TEST2	54	SELECT * FROM ZSQLM_TEST_TAB2 INTO TABLE ITAB2 FOR...	Batch Job	ZSQLM_T

SQLM data records

- Contain source position, request, table names
- For all SQL statements (740 SP5)

Copyright © 2022 Deloitte Development LLC. All rights reserved.

SQL Monitor—DB Performance Profile of the Business Processes

Business process list ranked by total DB time

SQL Monitor

- Displays performance data on process level (report, transaction, …)
- Allows to drill down from process level to the single DB operations

Application Help

Total DB Time	Total Time	DB T	Tot. DB E	Total Records	DB Mn T	Sessi	Request Type	Request Entry Point								
880.399.164,4	1.507.934.919,	58,38	148.713.640	1.018.292.1	5,920	527	Batch Job	ZSQLM_TEST								
454.645.987,6	530.479.364,2	85,70	74.191.995	495.636.853	6,128	6.287	Remote Func	ZSQLM_CALLBACK_RFC								
441.514.573,5	447.770.339,2	98,60	74.626.760	369.032.400	5,916	520	Batch Job	ZSQLM_TEST3	6	11.323,	0,376	16,223	100.000	0	4,945	651,831
144.984.946,4	156.231.746,6	92,80	236.797.836	2.840.340.7	0,612	3.156	Batch Job	ZSQLM_TEST11	8	214.15	0,300	13,951	75.000	0	11,995	908,016
14.423.850,765	14.484.963,817	99,58	69.432	473.415.780	207,741	3.156	Batch Job	ZSQLM_TEST11_OPT	3	12.440,	0,292	1.023,9	75.000	0	6.818,4	21.561,
3.492.813,580	58.553.884,853	5,97	22.360	119.797.600	156,208	520	Batch Job	ZSQLM_TEST3_OPT	5	9.865,5	10,521	729,088	100.000	0	5.357,6	20.941,
1.997.205,874	2.032.910,598	98,24	430.539	2.129.610	4,639	3	Submit Report	ZSQLM_TEST3	6	3.613,6	0,393	8,855	100.000	0	4,946	651,934
1.785.592,073	3.065.212,992	58,25	287.938	1.955.274	6,201	284	Unknown	INITIAL_ROOTID	225	7.015,5	0,144	22,117	100.000	0	6,791	751,869
1.137.344,454	1.165.832,179	97,56	618.893	6.993.684	1,838	9	Submit Report	ZSQLM_TEST11	3	547.32	0,340	748,489	75.000	0	11,300	889,157
715.917,334	899.285,486	79,61	262.216	969.077	2,730	22	Transaction	BC_TOOLS_USER	392	1.358,1	0,050	15,274	6.948	0	3,696	121,438
601.603,538	1.231.812,011	48,84	601.094	355.779	1,001	42.7	Batch Job	SAPMSSY2	94	4.638,6	0,174	7,507	38	0	0,592	0,563
590.491,030	602.329,819	98,03	139.673	730.590	4,228	2	Transaction	ZTRIGGER_TEST	43	1.141,8	0,306	5,786	100.000	0	5,231	660,846
348.633,241	1.373.833,686	25,38	345.576	2.135.053	1,009	10.5	Remote Func	SADT_REST_RFC_ENDPOINT	1.666	998,170	0,006	5,540	130.295	0	6,178	225,599
341.178,962	29.718.391,239	1,15	353.678	4.797.625	0,965	4.844	Submit Report	SAPMSSY8	184	1.061,1	0,178	3,408	3.926	0	13,565	58,993
340.130,032	695.698,023	48,89	30.326	12.289.442	11,216	404	Batch Job	RSQLM_UPDATE_DATA							5,244	1.124,2
339.255,716	405.406,056	83,68	111.829	10.664.854	3,034	525	Batch Job	ZSQLM_TEST2							5,368	20,866
291.224,558	519.537,938	56,05	57.200	37.076.000	5,091	520	Batch Job	ZSQLM_TEST4							8,182	2.247,8
197.292,681	343.344,709	57,46	106.665	1.204.334	1,850	38	Transaction	SEU_INT							1,291	195,367
193.693,937	224.064,019	86,45	83.002	271.595	2,334	526	Batch Job	ZSQLM_TEST1							3,272	1,251
173.430,426	302.778,616	57,28	158.909	2.414.562	1,091	236	Batch Job	SAPMS07A							15,195	269,917
133.813,181	990.688,638	13,51	71.728	2.538.910	1,866	238	Batch Job	SAPMSSYT	225	420,959	0,171	7,383	1.311	0	35,396	198,596
130.924,388	205.960,930	63,57	157.382	85.994	0,832	7.332	Batch Job	SWNC_TCOLL_STARTER	39	199,391	0,172	3,115	34	0	0,546	0,509
98.587,378																
92.503,767																
83.642,041																
80.660,881																
73.367,238																
67.243,635																
64.734,503																

Drill down to the SQL profile of report ZSQLM_TEST3 running as batch job (6 SQL statements)

Records for Request Batch Job ZSQLM_TEST3

Application Help

Total DB Time	DBTime[%]	DB Tm/TT	DB Exctns	DB Exct[%]	Total Records	Records[%]	DB Mn T	Table Names	SQL Operation Type	Obj.	Object Name	Include Name	Include Li	ABAP Source Code Fragment
438.268.499,380	99,26	97,88	74.620.000	99,99	5.137.600	1,39	5,873	ZSQLM_TEST_ORDER	SELECT (Open SQL)	PROG	ZSQLM_TEST3	ZSQLM_TEST3	59	SELECT SINGLE * FROM ZSQLM_TEST_ORDER INTO WA WHERE ORDER_ID = DETAILS
2.668.066,669	0,60	0,60	520	0,00	52.000.000	14,09	5.130,897	ZSQLM_TEST_ORDER	INSERT (Ope					
402.158,996	0,09	0,09	2.600	0,00	233.997.400	63,41	154,677	ZSQLM_TEST_ORDER	SELECT (Ope					
76.519,161	0,02	0,02	520	0,00	51.900.000	14,06	147,152	ZSQLM_TEST_ORDER	DELETE (Ope					
72.053,510	0,02	0,02	2.600	0,00	25.997.400	7,04	27,713	ZSQLM_TEST_ORDER	SELECT (Ope					
27.275,880	0,01	0,01	520						Commit					

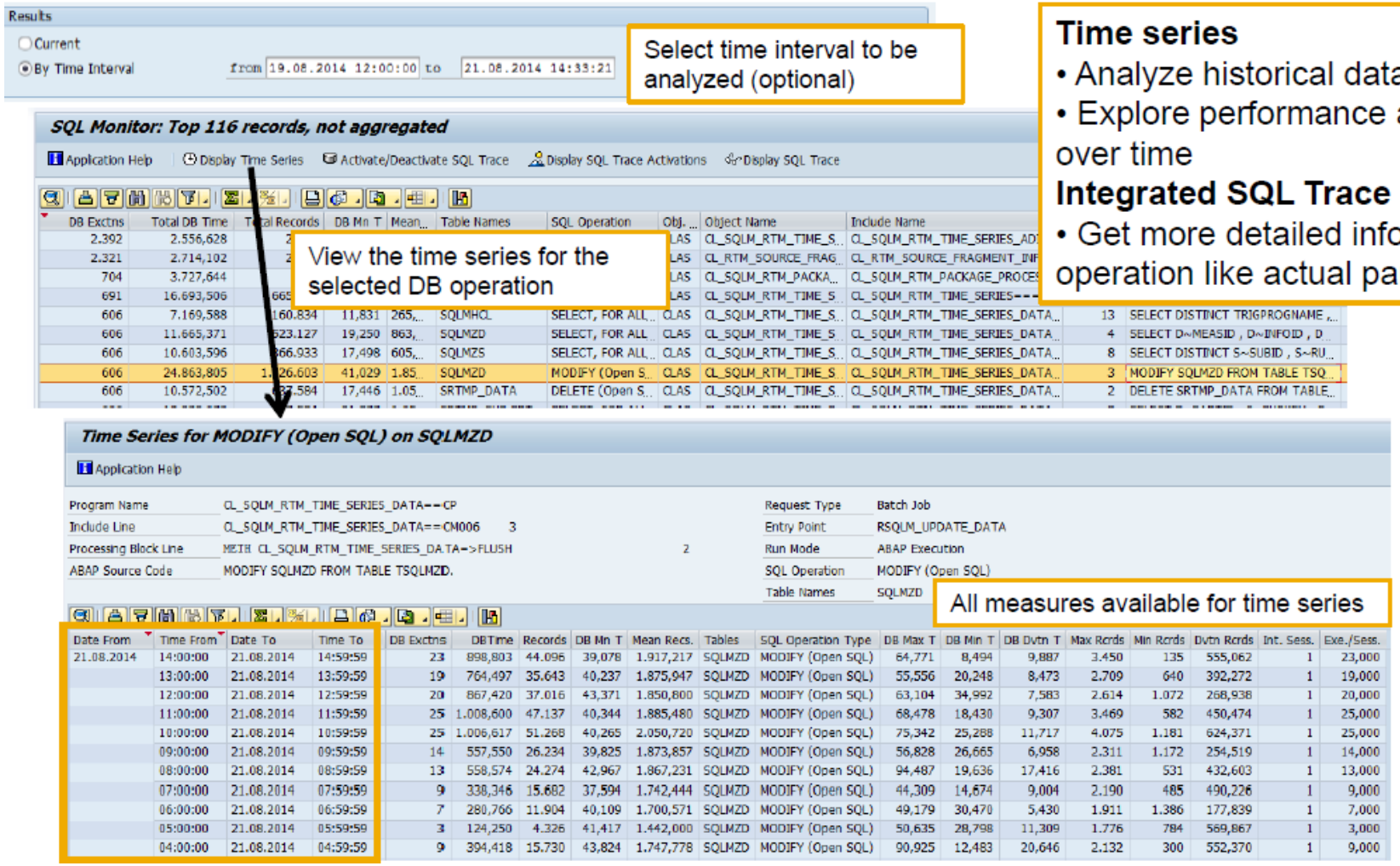
Navigate to the code

```
method get_details.
  data details type t_order_details.
  data wa type zsqlm_test_order.

  loop at important_order_details into details.
    select single * from zsqlm_test_order into wa
      where order_id = details-order_id and order_details_id = details-order_details_id and
            item_name like pattern. "check existence.
    if sy-subrc = 0.
```


Copyright © 2022 Deloitte Development LLC. All rights reserved.

SQL Monitor—LB Preview 740 SP8—Time Series and SQL Trace



Time series

- Analyze historical data
- Explore performance and usage of a DB operation over time

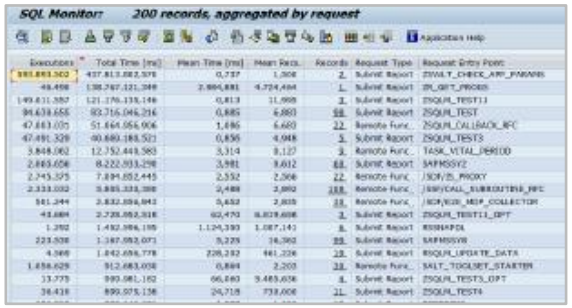
Integrated SQL Trace

- Get more detailed information for a selected ABAP DB operation like actual parameters and ABAP call stack

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Guided Performance Optimization

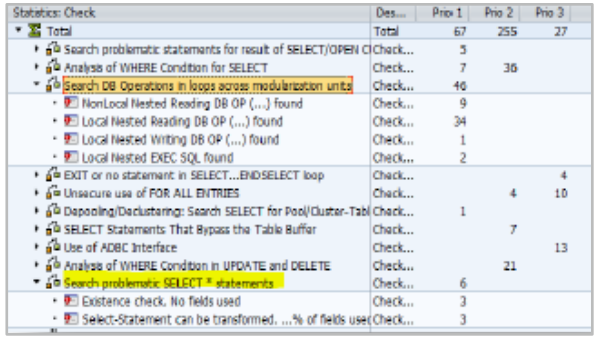
Runtime data



New SQL monitor (SQLM)

- Can run in production also before migration
- Captures aggregated runtime data (number of executions, database times, tables, transactions)

Static code analysis

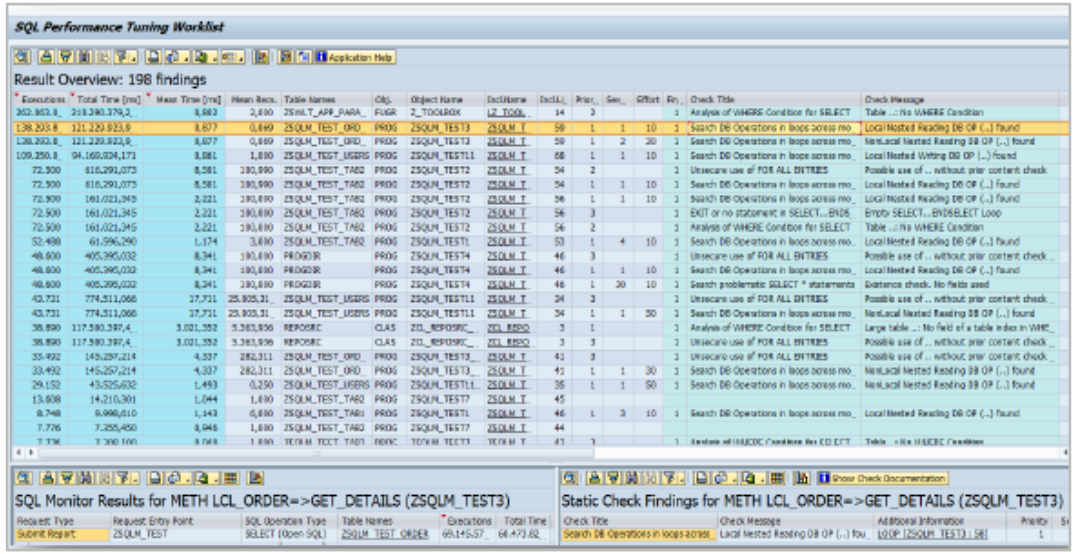


Improved/new code inspector checks, e.g.,

- Nested SELECT statements across modularization units
- Used columns vs. selected columns

SQL Performance Tuning Worklist (SWLT)

combination of SQLM and ATCH



Improved/new code inspector checks, e.g.,

- Define context from a business perspective (entry point, transaction, involved data)
- Fine code with the best cost/benefit ratio
- Fix performance bugs and “unexpected” database accesses
- Fine code which can be “pushed down” to SAP HANA (e.g. via SQL script procedure)

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Guided Performance Optimization—System View

Production System

Database

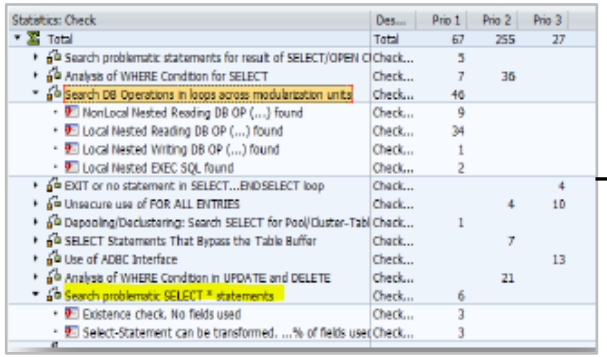
Development/ Q-System

Database

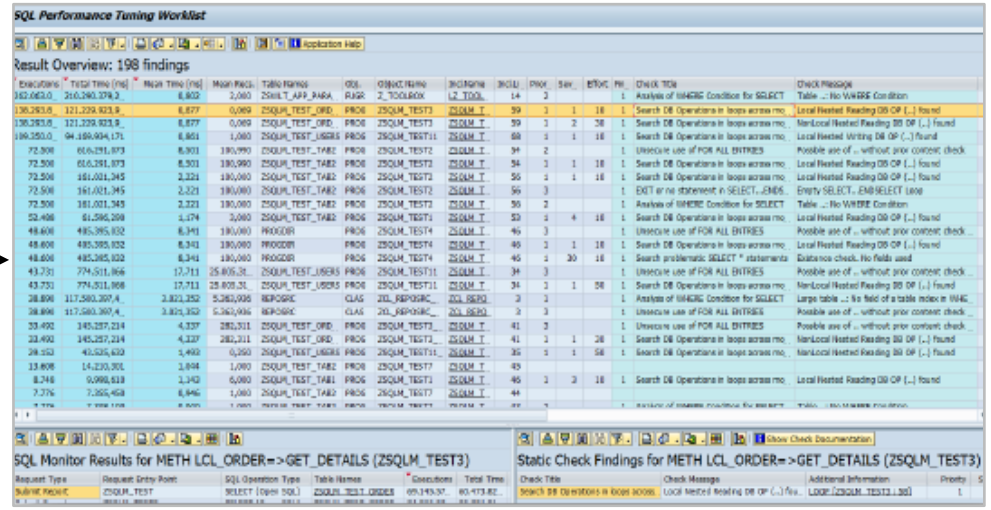
SQL Monitor

Download/RFC

ABAP Test Cockpit



SQL Performance Tuning Worklist



Copyright © 2022 Deloitte Development LLC. All rights reserved.

Process View—Tool Usage

Plan SAP HANA migration	SAP Business Suite Any Database	Upgrade Migrate	Sandbox System SAP HANA Database	Eliminate unused custom code to reduce migration effort Define your main business processes for testing and tuning Evaluate in sandbox system Use ABAP Test Cockpit to evaluate/correct potential functional issues in your custom code (“old” system or sandbox system) Use SQL Monitor in the “old” productive system (+SWLT in the development system) to tune the top custom SQL statements in your system
Prepare SAP HANA migration	SAP Business Suite Any Database	Upgrade Migrate	Development/Test System SAP HANA Database	Use ABAP Test Cockpit to correct potential functional issues and run ATC regularly to avoid regressions Use SQL Monitor and Runtime Checks in the SAP HANA test system (+ SWLT in the development system) to correct functional issues and to tune the top custom SQL statements
SAP HANA migration	SAP Business Suite Any Database	Upgrade Migrate	Production System SAP HANA Database	Use ABAP Test Cockpit for your quality assurance (avoid regressions) Use SQL Monitor and Runtime Checks in the SAP HANA productive system (+ SWLT in the development system) to correct findings and to iteratively tune the custom SQL statements Additionally, the SQL Monitor can help to detect and realize the potential for code pushdown to SAP HANA

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Summary

Here are the key learning points of the module.

- In general, only DB specific ABAP code must be analyzed, because existing ABAP code runs on SAP HANA as before.
- ABAP Test Cockpit can be used to find and adapt DB specific ABAP custom code easily
- The well-known golden Open SQL rules are still valid on SAP HANA, however, some priorities have shifted
- New monitoring tools like SQL Monitor are available to find HANA potential in existing ABAP code, and for effective SQL tuning.

STATIC CHECKS

1. ATC *USED FOR UPGRADE LEVEL CHECKS*

2. Code inspector

3. extended program checks

RUNTIME CHECKS

1. Performance trace (ST05). analysis related to only db

2. Runtime analysis (SAT). analysis related to both db and app layer. old tcode for it is se30.

3. SQL monitor *USED FOR UPGRADE LEVEL CHECKS*

Copyright © 2022 Deloitte Development LLC. All rights reserved.

Thank You

Deloitte.

About Deloitte
Deloitte refers to one or more of Deloitte Touche Tohmatsu Limited, a UK private company limited by guarantee ("DTTL"), its network of member firms, and their related entities. DTTL and each of its member firms are legally separate and independent entities. DTTL (also referred to as "Deloitte Global") does not provide services to clients. In the United States, Deloitte refers to one or more of the US member firms of DTTL, their related entities that operate using the "Deloitte" name in the United States and their respective affiliates. Certain services may not be available to attest clients under the rules and regulations of public accounting. Please see www.deloitte.com/about to learn more about our global network of member firms.

This communication contains general information only, and none of Deloitte Touche Tohmatsu Limited ("DTTL"), its global network of member firms or their related entities (collectively, the "Deloitte organization") is, by means of this communication, rendering professional advice or services. Before making any decision or taking any action that may affect your finances or your business, you should consult a qualified professional adviser.

No representations, warranties or undertakings (express or implied) are given as to the accuracy or completeness of the information in this communication, and none of DTTL, its member firms, related entities, employees or agents shall be liable or responsible for any loss or damage whatsoever arising directly or indirectly in connection with any person relying on this communication. DTTL and each of its member firms, and their related entities, are legally separate and independent entities.

Copyright © 2022 Deloitte Development LLC. All rights reserved.